# DeepSeek Harness 实测｜模型之外的那一半，到底带来了什么

腾讯技术工程 2026年8月14日 18:30 广东 143人



作者：osli元宝产品经理；相关团队：元宝产品中心、元宝模型工程中心、元宝大模型应用算法中心

DeepSeek-V4-Pro 正式版上线当天，DeepSeek Harness 也开源了。之前注册Deepseek Harness公众号已经吵闹过一阵。V4 Flash 发布报告的模型卡里只漏过一句 DSH，外界却已经猜测了很久，可谓期待值拉满了。

源码摊开以后，预期和实物略有点对不上号。很多人等的是 DeepSeek 版 Codex。当前开源的这套东西更像还能继续造 Agent 的 runtime。Web coding agent 有，Headless、Python SDK、ACP 和 JSON-RPC 也有。模型、工具、会话、权限、沙箱、Agent Loop 和 UI，都可以拆开再装。

我第一时间基于开源仓库，在本机把 npm、Web、Headless 和 Python SDK 都跑通了，拆开默认配置和 session log，又拿同一个 Kimi K3 对照 DSH 与 Kimi Code，最后让 V4 Pro 连续做一条带视觉产物的复杂任务。

跑完以后总体印象是：DSH 现在更像是一个开放的 Agent 平台脚手架，默认产品还带着预览版的毛边，留待社区的打磨。一切皆插件的理念挺新颖，Trajectory 的源码设计口碑不错，值得学习。

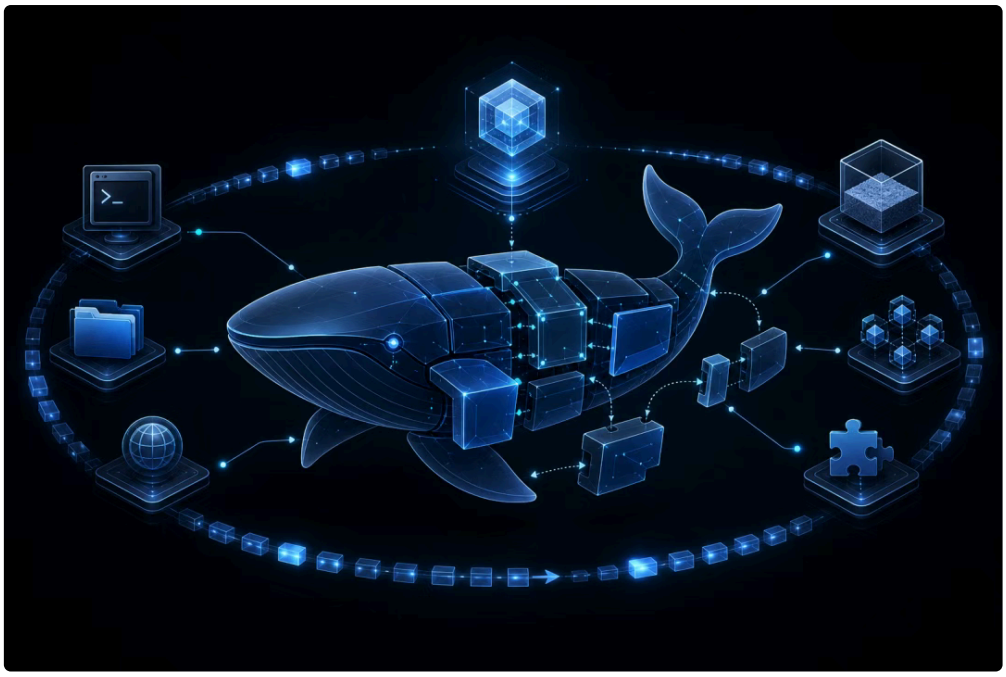


*图 1　按插件树、可替换能力和追加式事件流绘制的概念图。该图由图像模型生成，不是产品界面。*

DSH 仍处于 Developer Preview。文中数字来自单次本机运行和公开用户反馈，用来看差异和边界，不做产品排名。

## 社区从一开始就在看两样东西

知乎问题《如何评价在 8 月 13 日发布的 DeepSeek Harness》下的反馈，大致分成两组。

一组用户等的是成熟 coding agent。他们关心桌面 App、终端体验、Windows、缓存命中、日常编码效果，以及官方 Harness 能不能把 V4 Pro / Flash 再抬一截。

另一组内测者盯着插件、Preset、Trajectory 和运行时自修改。他们看见的是一个可以继续开发 Agent 的平台。

两边都没看错。DSH 同时有这两个身份，完成度却不对称。runtime 已经很开放，默认产品体验还没追上 Claude Code、Codex、Kimi Code。

### 最稳的好评是 Trajectory

段小草把 Trajectory 比作 Agent 的 DevTools。时间轴、每轮请求和日志都能看见，查上下文压缩、Skill 过多和模型犯错时很有用。

卜寒兮也特别提到运行信息面板。缓存命中、输入 / 输出 token 和 token 速度都直接显示出来。

这件事和源码对得上。DSH 要求模型看见的内容必须已经记进日志，Trajectory 直接从 session event log 投影。它没有另埋一套监控数据，所以看见的轨迹更接近模型当时收到的真实请求。

### 插件值钱的地方，是把 Agent 收窄

社区早期插件里有不少换肤和整活，更值得看的是领域 Agent。

Kitt 在进化分享的 Data Agent 只保留 read、edit、write，再用 `sqlcmd` 替换 bash。模型围着数据库执行结果转，同时不再带着和数据分析无关的工具与上下文。

这个例子把 Plugin 和 Preset 的分工讲清楚了。插件提供新能力，Preset 决定某类 Agent 能看见哪些能力。DSH 真有生产价值时，往往落在删掉什么、替换什么。继续堆工具，只会多耗 token，也让模型更难选。

### 长任务能跑，账单也要一起看

Adam Platin自述用一周内测版管理机器学习竞赛。Agent 读规则、跑脚本、推进实验并调用浏览器，89 步后收尾。

瓜子脸帅哥给出了另一条时间线。DSH 将 Agent 接入微信，从需求到真机收到回复约 87.6 分钟，花费约 18 元。过程包含 SDK 调研、零依赖客户端、mock server、真实端点冒烟和扫码绑定。

这两条都是用户自述，不能当 benchmark。它们仍然给出一个很实用的尺度。长任务是否做完只是一半，另一半是花了多少分钟、多少 step、多少 token、几次人工接管。

## 跑通不难，麻烦在上下文从哪进来

官方最快的路径是 `npx @deepseek-ai/dsh web`。Node 需要 `^22.19.0 || >=24`，启动后填写 API Key、选择工作区和 Agent Preset 就能用。源码安装还会碰到 pnpm、构建和 Git hooks，更适合准备开发插件的人。

本机实际跑通了四条入口。

- Web UI 能正常创建本地工作区与会话
- Headless 能执行一次性任务，成功时只向 stdout 输出最终回复
- Python SDK 自带捆绑运行时，不依赖系统 Node
- `--dump-config`
  可以直接打印最终插件树

默认配置也能直接核对。Web profile 组装出 129 行插件配置，Headless 为 81 行。默认模型是 DeepSeek V4 Flash，权限是 `workspace-write + ask`。不同模式不是界面开关，它们加载的提示词和工具集合确实不同。

### System Prompt 可以从日志还原

DSH 没有把运行时请求藏在内存里。System prompt 由 persona、工具说明和插件注册的 prompt sections 拼起来。发请求前，`request/header` 会记下 system、model、temperature、max tokens 与 tools，`request/context` 记下 provider 和 context window。

官方 Python SDK 的 minimal composition 更容易看清这个结构。它把 persona 固定成一句 `You are a helpful software engineer assistant.`，关掉 workspace prompt、

Skills 与 compaction，只留下 persistent bash 和 `str_replace_editor`。模型前缀被冻住了，工具面带来的干扰也少很多。

### Trajectory 底下是一条只能追加的事件流

会话以 zstd 压缩 JSONL 保存，每条记录都有统一外壳。

```
{"type": "tool/call", "seq": 31, "time": 1786632922876, "data": {}}
```

一次写文件任务在本机产生 61 条事件。里面有 `turn/start`、三个 `step/start`、用户消息、请求头、reasoning chunks、两次 tool call / result、assistant message 和 `turn/end`。模型先 write，再 read 核对，最后汇报。

所以 Trajectory 能同时回答三类问题。

1. 模型当时看见了什么
2. 哪一步调用了哪个工具
3. token、缓存和结束原因怎样变化

### 四个字母的任务，首包也能吃掉一万三Token

我用 DSH Headless 的默认配置做了一次最小测试，用户指令只有“回复 PONG”。尽管任务很短，首轮请求仍包含约 13,467 个 input token。继续检查日志后发现，主要开销来自默认加载的系统提示、工具说明、仓库规则和 Skill 摘要

测试目录位于一个大仓库内部。DSH 沿最近的 `.git` 找到仓库根，注入了 `AGENTS.md` / `CLAUDE.md`，又扫到用户技能目录里的 27 条摘要。写文件任务后续 step 出现约 1.3 万 cache-read token。这些固定前缀进了缓存，也说明它们确实进了模型请求。

后来评测和敏感项目，我都会换独立 Git 根，显式设置独立的 `DSH_HOME` 与 `DSH_AGENTS_HOME`。比较模型时用 minimal composition，这样才能够把上下文整理的干净一点。

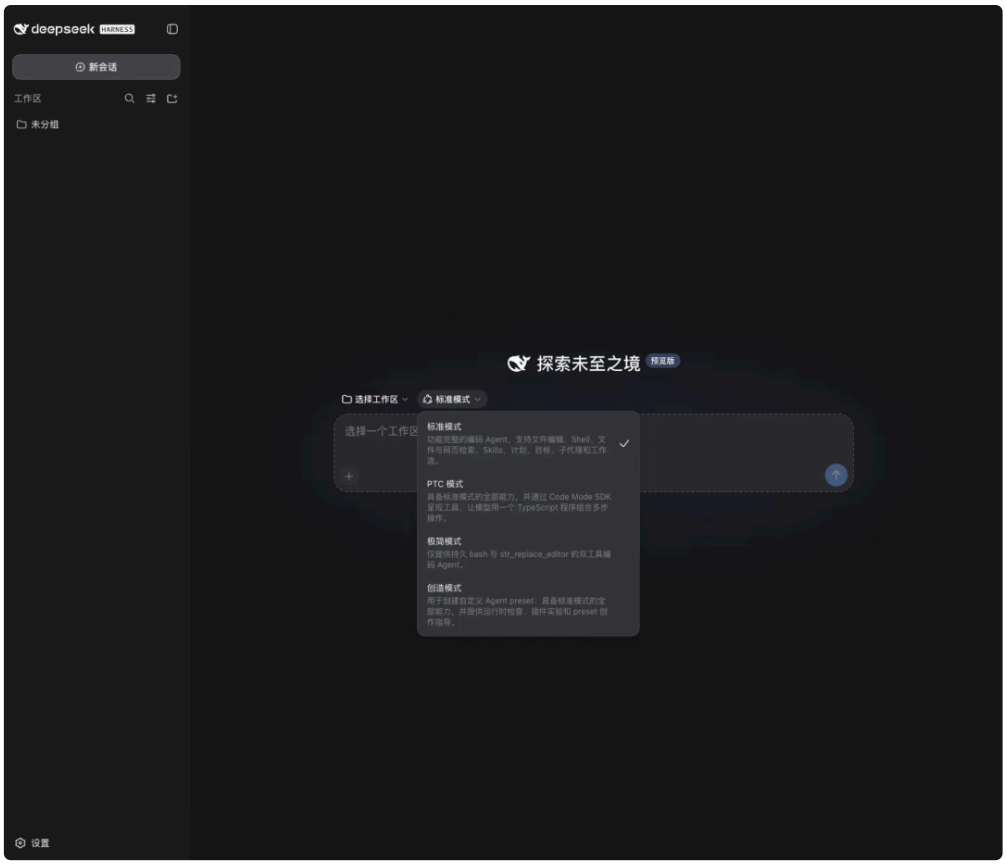



*图 2　标准、PTC、极简、创造四种预设对应不同插件组合。*

标准模式提供完整的编码工具，适合直接处理日常开发任务。PTC模式在此基础上允许模型用 TypeScript 组合多步工具操作，极简模式只保留 bash 和文本编辑器，创造模式则增加运行时检查、插件实验和自定义 Preset 所需的能力。

## 同一个 Kimi K3，换 Harness 后发生了什么

Kimi Code 与 DeepSeek Harness 都是国产、MIT 开源、可接多模型的 coding agent / Harness。两边的出发点并不相同。

| 维度 | DeepSeek Harness | Kimi Code CLI |
| --- | --- | --- |
| 官方定位 | 可重新组装的 Agent runtime | 开箱即用的终端 coding agent |
| 主要入口 | Web、Headless、Python SDK、ACP | TUI、`-p`、ACP、Web |
| 扩展方式 | Cordis Plugin、Preset、Patch、Skills、Hooks、MCP | Plugins、Skills、Hooks、MCP、Subagents |
| Agent Loop | 作为插件注册，架构上可替换 | v2 DI 中的 Agent-scope Service，源码开放但属于固定引擎主干 |
| 默认模型 | DeepSeek，另有 pi-ai 多 Provider | Kimi，支持兼容 Provider |
| 默认工具 | 标准模式约 25 项，含文件、Shell、任务、子代理、工作流、网页 | Read、Write、Edit、Grep、Glob、Bash、Web、Todo、Task / Subagent 等 |

| 维度 | DeepSeek Harness | Kimi Code CLI |
| --- | --- | --- |
| 权限 | read-only / workspace-write / danger-full-access | 常规审批、yolo、auto |
| 可观测性 | Session Event、Trajectory、token / cache、可编程投影 | stream-json、session export、`kimi vis` |
| 安装形态 | npm 需要 Node 22.19+ | 官方单文件分发无需 Node，npm 安装另需 Node |

Kimi Code CLI 的仓库本身也是开放的。两边都开源，差别在运行时怎么组织。Kimi Code 把可扩展能力围着一套终端产品转，DSH 把 loop、provider、session 和 UI 都放进同一种插件组合模型。

公开反馈普遍认为同模型换 Harness 会改变产物，但任务、权限和上下文经常没有保持一致。这里固定 Kimi K3 API、提示词、工作区与隐藏验收，看两套产品怎样完成相同任务。

- 模型统一使用 `kimi-k3`
- 提示词、任务模板和隐藏验收相同
- 每次使用全新 Git 工作区与空技能目录
- DSH 使用官方 minimal 工具面
- Kimi Code 使用 `0.35.0` 非交互模式
- 每题只有一次运行，因此只看轨迹

有一处我故意留了不同。DSH 使用 minimal，只暴露 persistent bash 与 `str_replace_editor`。Kimi Code 使用默认工具。这样不适合比较谁的工具更少，适合观察两套 Harness 怎样把同一个模型引向不同执行路径。若要比较默认产品能力，还得用 DSH standard 再跑一轮。

两道题分别是依赖批次规划与会话事件投影。每题有 7 个公开测试、8 个模型看不见的隐藏测试。

| 任务 | Harness | 验收 | 时间 | 轨迹单位 | 工具调用 |
| --- | --- | --- | --- | --- | --- |
| 依赖规划 | DSH minimal | 15 / 15 | 112.2 秒 | 9 step | 11 |
| 依赖规划 | Kimi Code | 15 / 15 | 48.9 秒 | 4 条 assistant 消息 | 5 |
| 会话投影 | DSH minimal | 15 / 15 | 111.3 秒 | 7 step | 7 |
| 会话投影 | Kimi Code | 15 / 15 | 123.1 秒 | 5 条 assistant 消息 | 6 |

正确性没有拉开。两边四条轨迹全部通过 15 / 15。第一题 Kimi Code 快约一分钟，第二题 DSH 快约十二秒，速度没有稳定赢家。

过程差得更清楚。

Kimi Code 首轮并行读取 README、源码和测试，随后整文件 Write，再用 Bash 验证。依赖规划只用了 5 次工具调用。它的默认工具更丰富，系统提示也更偏向尽快完成一个编码任务。

DSH minimal 主要在 persistent bash 与 `str_replace_editor` 之间来回走。依赖规划用了 11 次工具调用，步骤更碎。换来的是模型前缀、工具面和请求日志更容易冻住，session log 还给出完整 request、tool 与 usage 事件。

Kimi Code 的 `stream-json` 在本轮没有返回 token usage，因此不比较成本。DSH 两题分别记录 7,593 / 5,082 input token，以及 22,515 / 18,632 cache-read token。

> 同一个 Kimi K3 最终都过题，走法已经明显不同。Kimi Code 更像已经调好的成品。DSH minimal 更容易冻住前缀、换掉工具和 provider、把实验复现出来。

如果目标是日常终端编码，Kimi Code 的安装和默认工具更省事。如果目标是研究上下文、替换 provider、定制领域 Agent 或复现实验，DSH 的插件树与事件日志更好用。

### 源码层面，差别落在哪里

前面的轨迹差异，在源码里可以找到三组对应关系。

第一组是 Agent Loop。Kimi Code 当前默认使用 v2 引擎，把循环、工具调度和上下文管理组织成一组 DI Service，Plan、Swarm 等功能可以按需注入。DSH 则把 `ReactLoopAgent` 本身注册成 Cordis 插件，Loop 与模型 Provider、工具等服务采用同一种装卸方式。Kimi Code 的重心是一款可扩展的终端 Agent，DSH 留给开发者的改造范围更大，连 Agent 怎样循环都可以替换。

第二组是会话记录。Kimi Code v2 通过 `wire.jsonl` 保存 Agent 边界上的事件，DSH 也用 Session Event 恢复状态和驱动界面。DSH 还增加了一条运行时约束，模型看见的内容必须已经写进日志。这就是 Trajectory 能还原请求、工具调用和 token 变化的原因。

第三组是工具与执行环境。Kimi Code 用 Agent Profile 控制工具范围，再通过 KAOS 抽象本地与 SSH 环境。DSH 用 Preset 决定工具和提示词，再把文件系统、子进程和 sandbox 拆成可替换服务。两边都能换模型和执行位置，DSH 更强调更换这些底层服务时，上层工具保持不动。

所以，两套项目都有插件、事件日志和环境抽象，差别主要在开放到哪一层。Kimi Code 优先把终端编码体验调好，DSH 则把更多运行时部件交给开发者重新组合。这也解释了同一个 Kimi K3 在两边都能过题，执行步骤和日志形态却明显不同。

## 同一个 V4 Pro，两个 Harness 各做一款跳一跳

前面的两道小题适合看工具轨迹，还不足以观察 Harness 怎样影响完整产物。这一轮固定 `deepseek-v4-pro` API、Prompt 和验收要求，让 DSH 与 Kimi Code 在两个独立工作区里各做一款原创跳一跳游戏。

任务不只要求画出一个网页。玩家要能用鼠标、触摸和空格蓄力起跳，游戏要处理平台碰撞、Perfect 奖励、连击、失败与重开。项目还要带一个零依赖 Node.js 后端，提供排行榜、统计、JSON 持久化、输入校验和路径防护，并留下确定性测试接口。参考 VISTA 的思路，最终验收同时看代码测试、真实浏览器行为和画面。

先看最终结果。Kimi Code 做成了全屏 2.5D 方块舞台，DSH 做成了 2D 横向画布，右侧放排行榜和操作说明。




*图 3　同一模型、同一 Prompt 的最终游戏界面。左侧为 Kimi Code，右侧为 DSH，均来自真实浏览器运行态。*

两个页面均为前端单文件版本，游戏可以直接运行；排行榜与统计功能仍需配套 Node 后端。

我自己也各玩了几局。两个版本都不只是能打开，蓄力、落点和连续跳跃已经有了小游戏该有的手感。Kimi 版的舞台感更强，DSH 版更像一个带完整信息区的 Web 游戏。

两套 Harness 都完成了前端、后端、排行榜持久化和自动测试，过程却不一样。DSH 分三段 session 完成，合计约 25 分钟、85 个 step 和 96 次工具调用，交付 19 项后端测试。第一次运行中断后，它能够接着补齐约 880 行游戏逻辑和浏览器 smoke test。

Kimi Code 初次生成约 14 分 30 秒，算上两轮修复约 21 分钟，交付 17 项后端测试。它的初版已经有完整的 2.5D 视觉风格，后续根据真实浏览器反馈修复了 `updateHUD` 调用和遮罩层显示问题。

这次评测留下三点印象。

第一，同一个模型和同一份 Prompt，最终仍然长成了两种产品。Kimi Code 更偏向全屏游戏体验，DSH 更偏向带排行榜和说明区的完整页面。小样本无法证明这是稳定倾向，但足以说明 Harness 会参与塑造产物。

第二，两边都能承载二十分钟左右、有前后端、有测试、需要中途修复的任务。DSH 的固定物理步长、seeded PRNG 和原子写入，Kimi Code 的 2.5D 舞台与完整交互，都已经超过

一次性页面 Demo。

第三，两边都出现过模型自测通过、真实浏览器失败。DSH 的 canvas backing store 尺寸错误，画面只剩天空。Kimi Code 先遇到函数归属错误，随后又被 CSS 覆盖了 `hidden` 属性。问题反馈给 V4 Pro 后都修好了，也给各自的浏览器测试补上了回归检查。

这条共同的失败最值得记住。Agent 写的测试很容易和实现共用盲点。Harness 可以保留完整轨迹，也能帮助模型继续修正，最终仍要靠独立浏览器、外部验收和人眼试玩来确认产物真的可用。

## DSH 也能驱动其他模型

DSH 同时提供 DeepSeek 直连适配器和基于 pi-ai 的多 Provider 适配器。本机分别使用 Kimi K3、GPT-5.6 Sol 和 Claude Opus 4.8 完成了文件写入、读回与工具调用。

这项冒烟只说明 provider、OpenAI-compatible 消息、SSE 和工具协议可以工作，不用来比较三种模型。实际接入仍应逐个验证文本、流式、工具调用、错误映射和长连接。

## 这些体验在源码里从哪来

仓库采用 TypeScript monorepo。主干可以先压成这几层。

```
deepseek-harness/
├── apps/cli + apps/web          命令行与浏览器入口
├── packages/boot + bundle       Profile 与插件树组装
├── packages/core/
│   ├── agent-loop               Turn / Step 驱动
│   ├── session                  追加式事件日志
│   ├── tools                    工具注册与执行
│   └── system-prompt            Prompt 片段组装
├── packages/preset              每会话 Agent 组合
├── packages/llm                 DeepSeek 与多 Provider 适配
├── packages/fs + sandbox        可替换执行能力
├── packages/code-runtime        PTC / Code Mode
└── vendor/cordis                插件生命周期底层
```

这张图能解释 DSH 的技术栈。Cordis 管插件生命周期和服务依赖，DSH 在上面定义 Agent 领域的 session、loop、tools、LLM、sandbox 和 UI。KM 的运行时拆解对 Fiber、Preset、Code Mode 与工具遮蔽有更细的源码映射，这里只留和使用体验最相关的五点。

### 一切皆插件，靠的是 Cordis 的可逆生命周期

代码入口在 `vendor/cordis/src/context.ts`、`fiber.ts` 与 `service.ts`。

插件通过 `inject` 声明依赖的服务。依赖没出现时，Fiber 停在 `PENDING`。服务出现后进入 `ACTIVE`。提供方退出时，依赖组件先卸载，再回收自身 effect。

每次注册服务、监听器或定时器，都要把 disposer 一起交给 `ctx.effect()`。卸载时按相反顺序清理。Agent Loop 能成为普通插件，前提正是底层已经能管住这种体量组件的加载、依赖和退出。

这个机制也有明确边界。外部文件、网络消息和已经发生的业务动作不会自动回滚。依赖注入也无法替代恶意代码沙箱。

### Profile 与 Preset 分别控制进程和会话

代码入口在 `packages/boot/app-boot/src/profile.ts` 与 `packages/preset/agent-presets/src/`。

Profile 从空列表开始叠 bundle，再应用 profile、home 和命令行 patch。Web 与 Headless 因此是两棵不同的插件树。`dsh --profile web --dump-config` 使用同一套 patch 算法输出最终结果，配置展示和真实启动不会各维护一套逻辑。

Preset 再决定某个会话看见哪些工具、提示词和投影单元。同一个进程里可以同时跑 standard、minimal 或领域 Agent。前面 Data Agent 用 `sqlcmd` 替换 bash，就是这套作用域落到产品上的例子。

### 模型看见的内容，必须能从日志重建

代码入口在 `packages/core/agent-loop/src/agent.ts`、`invariant.ts` 与 `packages/core/session/src/`。

每一步请求都从 session 派生 messages。请求发出前，invariant 会再比较当前请求与 `session.deriveMessages()`，并核对模型、system prompt、temperature、max tokens 和 tools。

```
const expected = session.deriveMessages()
if (JSON.stringify(options.messages) !== JSON.stringify(expected)) {
  fail('log-reconstruction desync')
}
```

这条约束解释了 Trajectory 为什么清楚，也解释了 fork、恢复和 UI 为什么能共用同一条事件流。它还带出一条隐私含义。上下文一旦注入模型，也会进入日志和数据边界。

### Capability Seam 把工具与执行环境拆开

代码入口在 `packages/fs/fs/src/types.ts`、`fs-local/`、`fs-sandbox/` 与 `tool-str-replace-editor/`。

文件系统先定义 `FileSystem`、opaque target 和 version，再由 local / sandbox provider 实现，最后由 read、write、editor 消费。把文件与子进程 provider 换到远程沙箱时，上层工具不必跟着 fork。

写入还支持 `createIfAbsent` 与 `replaceIfVersion`。模型读完文件以后，若文件已被别人改过，provider 可以拒绝陈旧覆盖。并发安全因此落在工具下方，不必由每个编辑工具再写一遍。

### PTC 用代码减少工具往返

代码入口在 `packages/code-runtime/`。

PTC 让模型写一段 TypeScript 来组合多次工具调用。程序在 `worker_threads` 中执行，工具请求通过消息通道回到宿主，仍然经过同一套 pre-execute、审批、调度和 post-execute 流水线。

中间数据可以留在运行环境里，不必每一步都塞回模型上下文。这比模型会写代码更要紧。代价是执行模型生成的代码需要额外隔离，worker 仍不是完整安全边界。

## 现在谁值得花时间

做 Agent 基础设施的人值得马上看。DSH 把 loop、session、provider、工具和 UI 都摊在能核对的源码里。

需要领域 Agent 的团队值得做小实验。先复制 minimal / standard preset，再删工具、换 provider，比迁移整套工作流更容易验证收益。

模型评测人员可以用 minimal composition。同时锁定模型版本、推理档位、endpoint 和外部验收。

只想找日常编码工具的人可以先观望。Kimi Code、Claude Code、Codex 等产品在终端体验、桌面入口、IDE 集成和默认工作流上更成熟。

生产接入要自己补治理。插件来源、配置 diff、凭证边界、Windows 验收、日志保留和数据外发，都得部署方承担。

如果团队正在设计自己的 Harness，还可以带走三个问题。

1. 运行时最终加载了什么，能不能一条命令打印出来
2. 模型实际看见了什么，能不能从日志完整重建
3. 换掉文件系统、沙箱或模型提供方时，有多少工具必须跟着改

这三个问题，DSH 已经给出能跑、能核对的答法。默认产品、插件质量和治理能不能追上这套结构，是下一阶段的事。

---

## 资料汇总

### 官方与源码

- DeepSeek Harness 源码
- 中文架构文档

- Python SDK 最小 Agent 指南
- Cordis 源码
- 时空可组合性论文
- V4 Pro 0813 模型卡
- Kimi Code CLI 源码

### 用户反馈与方法

- 知乎目标问题
- 一周内测与 89 步竞赛任务
- 默认工具与 token 开销估算
- 87.6 分钟接入微信记录
- Data Agent 的 Preset 实践
- VISTA 视觉 Web App Benchmark

### 本地复现材料

- 基础结果 `working/eval/results.md`
- Kimi K3 对照 `working/cross-model/harness-compare/`
- Kimi Code 源码快照 `working/kimi-code/`
- 多模型冒烟 `working/cross-model/compat/`
- V4 Pro 跳一跳 Case `working/jump-game/workspace/`
- 知乎反馈摘录 `working/eval/zhihu-feedback.md`
- KM 图片与 HTML 附件 `output/attachments/`

全部数字以 2026-08-14 的单次本机运行结果为界。

📦 **完整复现材料获取方式：**
关注「腾讯技术工程」公众号，在后台私信关键词 **「deepseek」**，即可获取。




**腾讯技术工程**

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号







阅读 1.6万　修改于2026年8月15日

**留言**

写留言

还没有留言，来写首评